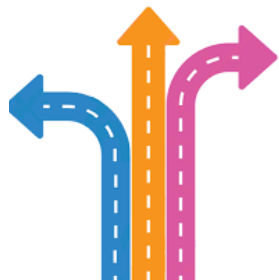


Criterion A: Examples



Criterion A is the foundational stage of the Internal Assessment (IA) and corresponds to the Problem Specification component of the Computational Thinking Process. This section establishes the overall requirements and scope that will govern the design, development, and evaluation of your entire solution.

Extended Examples for Criterion A Documentation



The following examples illustrate how to meet the Level 3–4 requirements by linking the scenario, measurable requirements, success criteria, and context in a concise manner.

Example 1: Object-Oriented Stand-alone Solution (e.g., Financial Tracker)

Project Context: Developing a personal finance tracker.

Section	Content Example (Aimed at Level 3–4)
Problem Scenario	The current situation is that students within my local school community, particularly those managing their own expenses in multiple currencies (e.g., during IB trips or study abroad), rely on fragmented spreadsheet tools. This causes problems because manual calculation and non-centralised expense tracking frequently lead to budget overruns and inaccurate savings projections. The primary users are students (16–18 years old) who need clear, automatic consolidation of income and expenses.

Section	Content Example (Aimed at Level 3–4)
Measurable Requirements & Objectives	The objectives are to centralise expense data and provide predictive reporting. The solution must handle transactional data from multiple categories and calculate running totals. The core requirement is the automated comparison of actual monthly spending against a user-defined budget, highlighting any deficit or surplus.
Success Criteria (Measurable)	A.1: The solution must store all transactional data persistently in a local file (e.g., CSV/JSON) and successfully load all records upon startup. A.2: The calculation engine must accurately flag a budget overrun (deficit) if spending exceeds the user's defined limit for any given month. A.3: The program must allow the user to filter transactions by date range and expense category within 500 milliseconds. etc.
Computational Context	I have chosen a stand-alone Python application using Object-Oriented Programming (OOP). This context is suitable because the primary complexity lies in building reusable classes (Transaction, Budget) and developing robust algorithmic logic for filtering, sorting, and calculation, rather than distributed web access. Python facilitates rapid development and efficient local data manipulation through libraries such as csv or json for persistent data storage, datetime - managing time-based transactions, and pandas - organising, filtering expense data.

Example 2: Database Management System (e.g., Library Management)

Project Context: Designing a library management system.

Section	Content Example (Aimed at Level 3–4)
Problem Scenario	My local community library currently uses outdated paper-based ledgers to track book check-outs. This presents problems in accurately determining book availability, calculating late fees, and identifying lost resources. The affected parties are the library staff, who waste significant time manually searching records, and the patrons, who receive inaccurate information. The general requirement is a centralised system to manage inventory and patron lending history.
Measurable Requirements & Objectives	The core objective is to ensure data consistency and integrity for book and patron records. The solution requires a permanent, scalable storage structure to facilitate complex querying, such as identifying patrons with overdue books or calculating resource availability.
Success Criteria (Measurable)	A.1: The system must prevent the addition of duplicate book titles using a primary key constraint. A.2: The querying mechanism must return a list of all patrons with loans overdue by more than 30 days within 1 second. A.3: The system must accurately update the LendingStatus flag in the

Section	Content Example (Aimed at Level 3–4)
	database whenever a book is checked in or checked out, ensuring transactional consistency. etc.
Computational Context	I have chosen a database solution (using SQL) managed via a Python interface. This context is required because the problem demands data manipulation and the use of relational structures (Theme A3) to maintain data integrity and handle complex logic conditions over a persistent set of records. The system will use SQLite as a file-based relational database, ensuring persistent local storage without an external server. SQL queries will be executed by the sqlite3 library, and the pandas library may be used to structure and query tabular data efficiently during analysis or reporting.

Example 3: Algorithmic Security Utility (e.g., Encryption Tool)

Project Context: Building a simple encryption/decryption utility.

Section	Content Example (Aimed at Level 3–4)
Problem Scenario	A group of students collaborate on sensitive academic projects and share notes via cloud storage, where data security is a concern. The problem is that the information is stored as plain text, increasing the risk of unauthorised access or interception. The goal is a pre-emptive security utility to render text unreadable without a specific key. This requires applying algorithmic techniques to secure information.
Measurable Requirements & Objectives	The main objective is to implement a robust, student-designed cipher (e.g., substitution cipher) that reliably transforms plaintext into ciphertext and back again. The solution must demonstrate clarity in algorithmic thinking and decomposition.
Success Criteria (Measurable)	A.1: The program must successfully encrypt and decrypt a 100-character text file, restoring 100% of the original content without error. A.2: The encryption function must process the file content in less than 50 milliseconds. A.3: The decryption key validation method must reject incorrect keys and prevent unauthorised data restoration. etc.
Computational Context	I will use a stand-alone Java console application. Java provides a suitable environment for focusing on the efficiency and clarity of core algorithmic design (Theme B2) required for the cipher logic. The choice of a console interface simplifies the problem, allowing maximum focus on the computational challenge of encryption and decryption.

Section	Content Example (Aimed at Level 3–4)
	For implementation of the crucial algorithms, I will use Java's libraries such as: • java.nio.file for reading and writing files containing plaintext and ciphertext. • java.util for managing input data, random number generation (Random, SecureRandom) • java.time for measuring the execution time of encryption/decryption. • java.lang – built-in string manipulation (StringBuilder, charAt(), substring()).

All materials on this website are for the exclusive use of teachers and students at subscribing schools for the period of their subscription. Any unauthorised copying or posting of materials on other websites is an infringement of our copyright and could result in your account being blocked and legal action being taken against you.

 **Report a problem**